

인공지능수학 (2026-1)

## 8. 분류와 클러스터링 (1)

국립금오공과대학교 컴퓨터공학부  
컴퓨터공학전공 / 인공지능공학전공  
김 경 수

# 목차



- I. K-NN algorithm
- II. K-NN algorithm 종합 예제
- III. K-NN algorithm의 주요 이슈

- ① K-NN 알고리즘의 정의와 구성 요소에 대하여 이해하고, 이를 논리적으로 설명할 수 있다.
- ② K-NN 알고리즘을 이용하여 주어진 데이터를 분류하는 방법과 이를 위해 벡터 거리 계산 및 유사도 측정 공식이 어떻게 활용되는지 이해할 수 있다.
- ③ 실제 간단한 사례를 이용하여 K-NN 알고리즘의 동작 원리를 이해하고, 이를 프로그래밍 언어를 이용하여 구현할 수 있다.

# K-NN algorithm 개요

# K-NN algorithm 소개



- **K-NN : K-nearest neighbor**
- 입력 데이터가 벡터로 주어졌을 때, 해당 입력 데이터가 어떤 클래스에 속하는지 판별하기 위한 알고리즘.
- 기존에 존재하는  $m$ 개의 데이터 중에서 입력 데이터와 가장 가까운  $k$ 개의 데이터가 어떤 클래스에 속하는지 분석하고, 이에 따라 입력 데이터의 클래스를 판단한다.

# Notation

- $i$ 번째 인스턴스:  $\vec{x}^{(i)} = [x_1^{(i)}, x_2^{(i)}, \dots, x_n^{(i)}]^T \in \mathbb{R}^n$
- 새로운 인스턴스:  $\vec{x}^{(q)} = [x_1^{(q)}, x_2^{(q)}, \dots, x_n^{(q)}]^T \in \mathbb{R}^n$
- 새로운 인스턴스의 label:  $f(\vec{x}^{(q)}) \in V$  or  $f(\vec{x}^{(q)}) \in \mathbb{R}$
- $i$ 번째 인스턴스의 label:  $\forall_{1 \leq i \leq m}, f(\vec{x}^{(i)}) \in V$  or  $f(\vec{x}^{(i)}) \in \mathbb{R}$
- 인스턴스를 label로 사상(mapping)시키는 함수:  $f : \mathbb{R}^n \rightarrow V$  or  $f : \mathbb{R}^n \rightarrow \mathbb{R}$
- 두 인스턴스 간의 거리(distance):  $D(\vec{x}^{(q)}, \vec{x}^{(i)})$

# K-NN algorithm의 수행 절차

- 새로운 인스턴스  $\vec{x}^{(q)}$ 가 입력된다.
- $\vec{x}^{(q)}$ 와 기존에 존재하는 인스턴스와의 거리(유사도)를 계산한다.
- $\vec{x}^{(q)}$ 와 거리가 가장 가까운(또는 가장 유사한) 인스턴스  $k$ 개를 선택한다.
  - 이를  $\vec{x}^{(1)}, \vec{x}^{(2)}, \dots, \vec{x}^{(k)}$ 라 하자.
- $k$ 개의 인스턴스들이 속한 클래스 중에서 이들을 가장 많이 포함하는 클래스를  $\vec{x}^{(q)}$ 의 클래스로 지정한다.

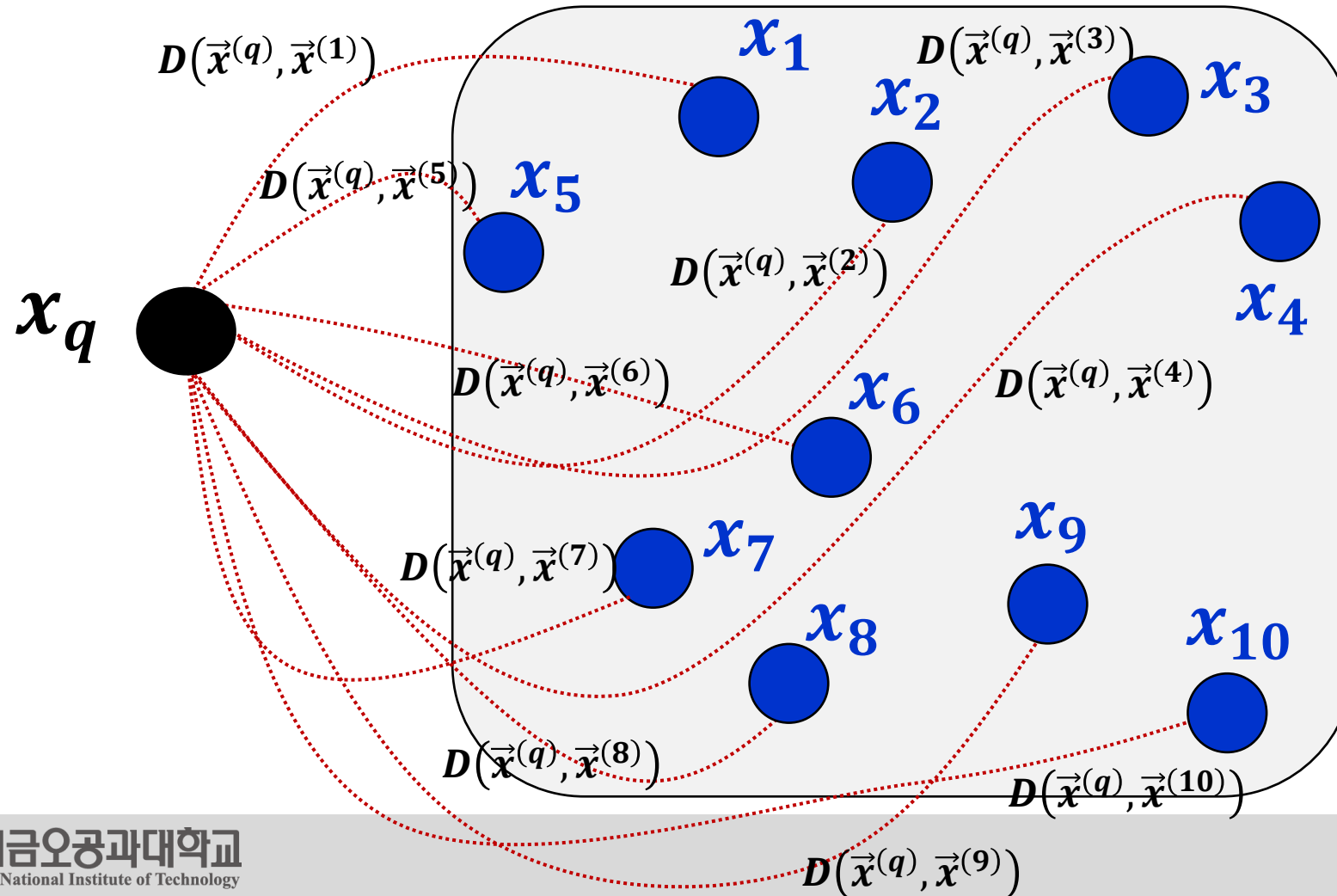
$$f(\vec{x}^{(q)}) = \arg \max_{v \in V} \sum_{j=1}^k \delta(v, f(\vec{x}^{(j)}))$$

- 위 수식에서  $\delta(a, b) = 1$  if  $a = b$  and  $\delta(a, b) = 0$  otherwise.

\*  $\delta(a, b)$ : Kronecker delta

# K-NN algorithm의 수행 절차

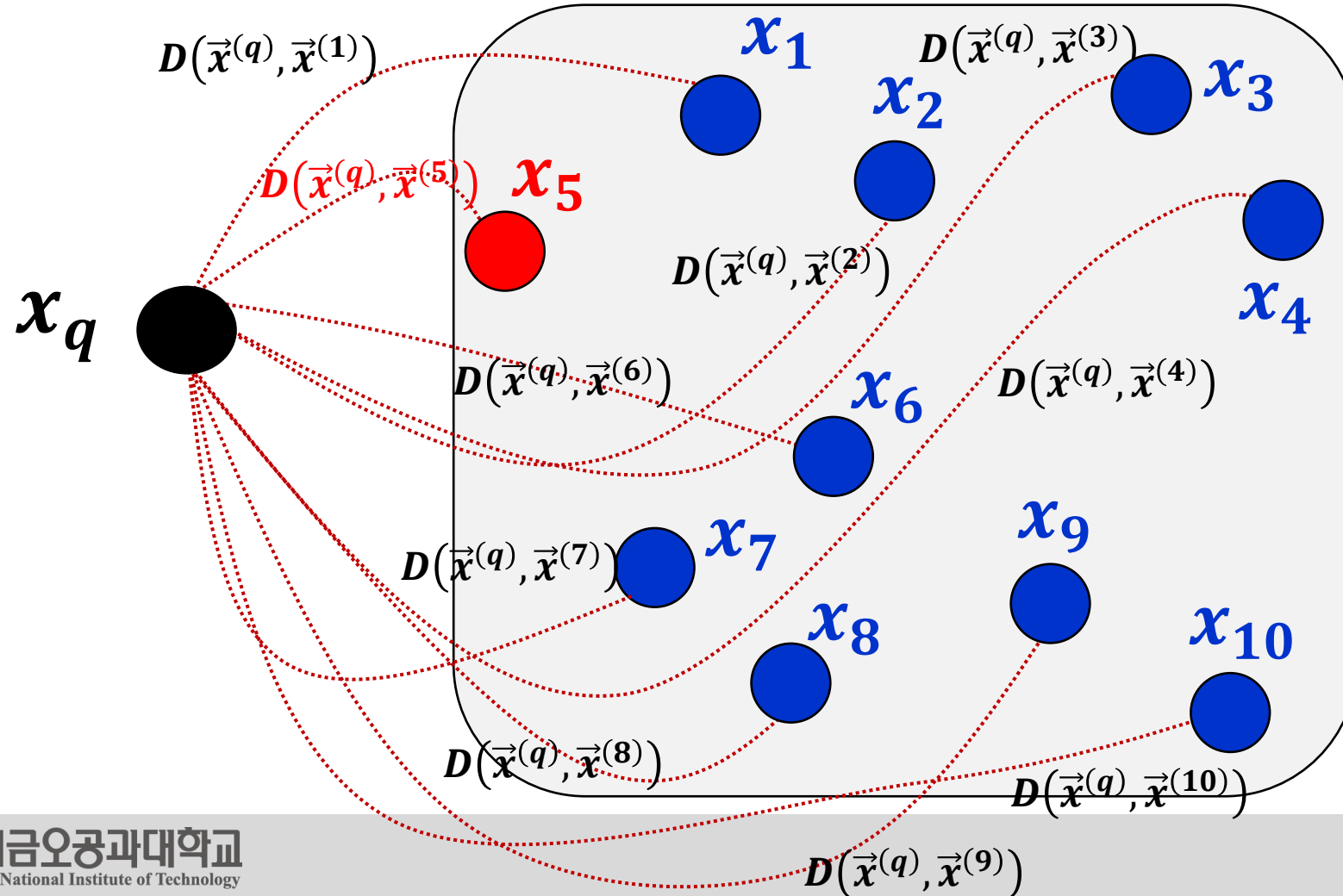
## • Example



| Inst.            | Class                   |
|------------------|-------------------------|
| $\vec{x}^{(1)}$  | $f(\vec{x}^{(1)}) = 1$  |
| $\vec{x}^{(2)}$  | $f(\vec{x}^{(2)}) = 2$  |
| $\vec{x}^{(3)}$  | $f(\vec{x}^{(3)}) = 3$  |
| $\vec{x}^{(4)}$  | $f(\vec{x}^{(4)}) = 1$  |
| $\vec{x}^{(5)}$  | $f(\vec{x}^{(5)}) = 3$  |
| $\vec{x}^{(6)}$  | $f(\vec{x}^{(6)}) = 2$  |
| $\vec{x}^{(7)}$  | $f(\vec{x}^{(7)}) = 2$  |
| $\vec{x}^{(8)}$  | $f(\vec{x}^{(8)}) = 1$  |
| $\vec{x}^{(9)}$  | $f(\vec{x}^{(9)}) = 3$  |
| $\vec{x}^{(10)}$ | $f(\vec{x}^{(10)}) = 2$ |

# K-NN algorithm의 수행 절차

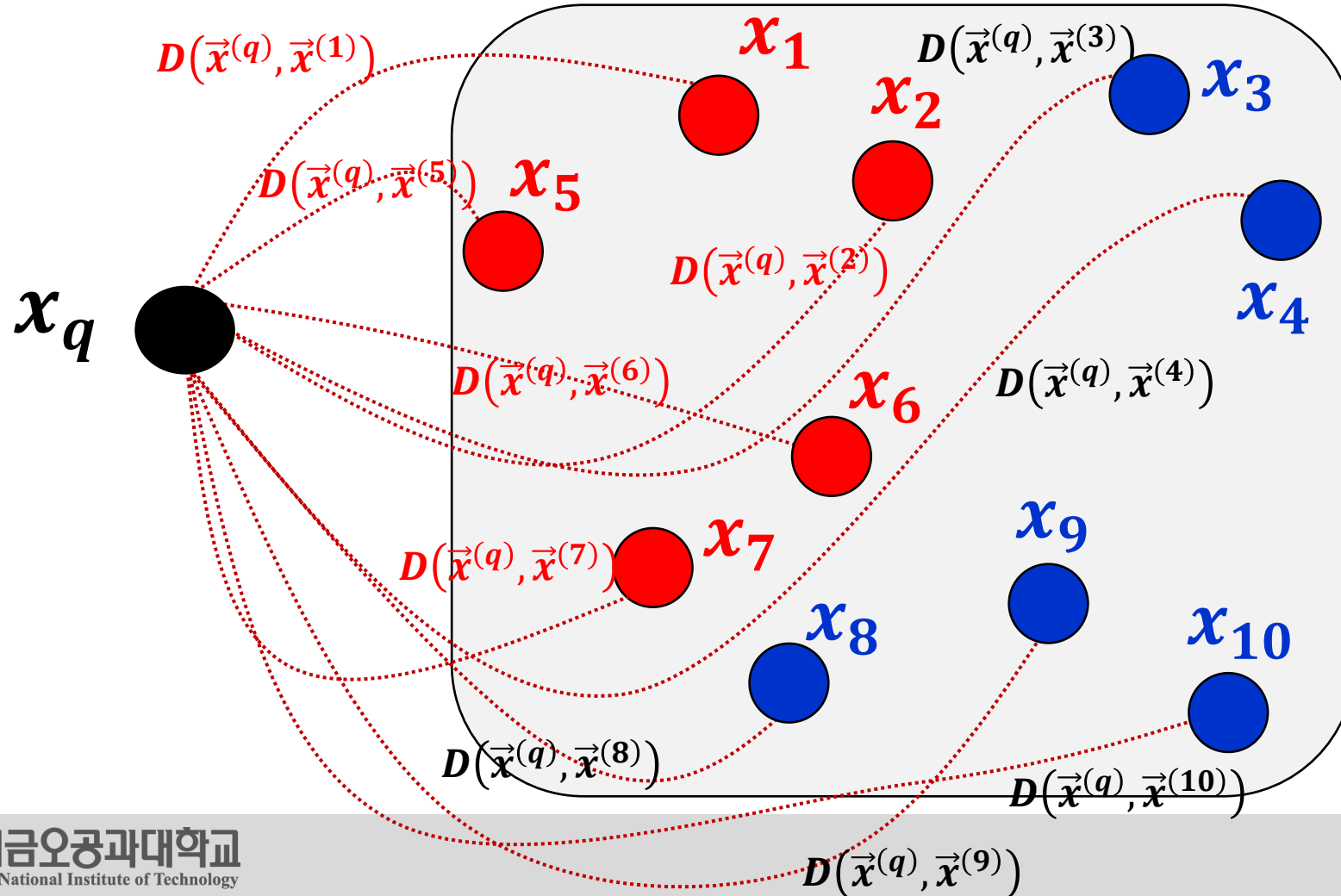
- Example (k=1)



| Inst.            | Class                   |
|------------------|-------------------------|
| $\vec{x}^{(1)}$  | $f(\vec{x}^{(1)}) = 1$  |
| $\vec{x}^{(2)}$  | $f(\vec{x}^{(2)}) = 2$  |
| $\vec{x}^{(3)}$  | $f(\vec{x}^{(3)}) = 3$  |
| $\vec{x}^{(4)}$  | $f(\vec{x}^{(4)}) = 1$  |
| $\vec{x}^{(5)}$  | $f(\vec{x}^{(5)}) = 3$  |
| $\vec{x}^{(6)}$  | $f(\vec{x}^{(6)}) = 2$  |
| $\vec{x}^{(7)}$  | $f(\vec{x}^{(7)}) = 2$  |
| $\vec{x}^{(8)}$  | $f(\vec{x}^{(8)}) = 1$  |
| $\vec{x}^{(9)}$  | $f(\vec{x}^{(9)}) = 3$  |
| $\vec{x}^{(10)}$ | $f(\vec{x}^{(10)}) = 2$ |

# K-NN algorithm의 수행 절차

- Example (k=5)

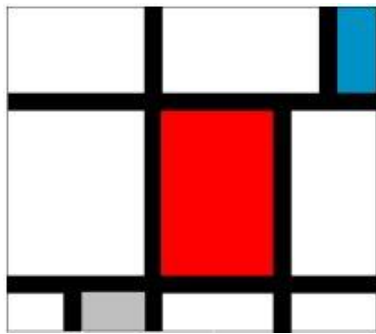


| Inst.            | Class                   |
|------------------|-------------------------|
| $\vec{x}^{(1)}$  | $f(\vec{x}^{(1)}) = 1$  |
| $\vec{x}^{(2)}$  | $f(\vec{x}^{(2)}) = 2$  |
| $\vec{x}^{(3)}$  | $f(\vec{x}^{(3)}) = 3$  |
| $\vec{x}^{(4)}$  | $f(\vec{x}^{(4)}) = 1$  |
| $\vec{x}^{(5)}$  | $f(\vec{x}^{(5)}) = 3$  |
| $\vec{x}^{(6)}$  | $f(\vec{x}^{(6)}) = 2$  |
| $\vec{x}^{(7)}$  | $f(\vec{x}^{(7)}) = 2$  |
| $\vec{x}^{(8)}$  | $f(\vec{x}^{(8)}) = 1$  |
| $\vec{x}^{(9)}$  | $f(\vec{x}^{(9)}) = 3$  |
| $\vec{x}^{(10)}$ | $f(\vec{x}^{(10)}) = 2$ |

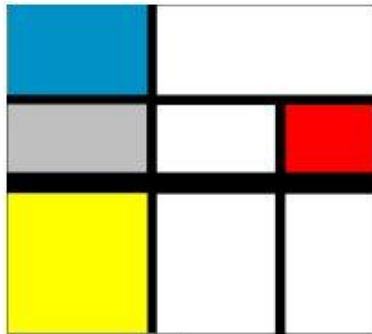
# K-NN algorithm 종합 예제

# K-NN algorithm 예제

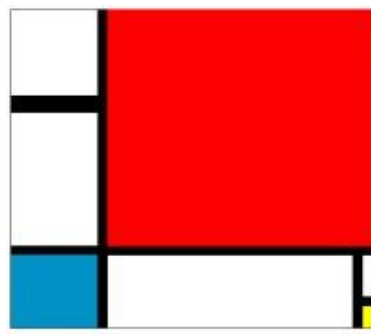
- 아래와 같은 몬드리안(Mondrian) 그림에 대한 이진 분류를 수행해 보자.



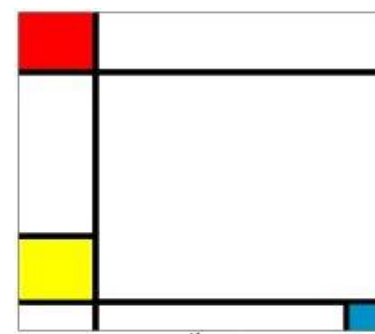
one



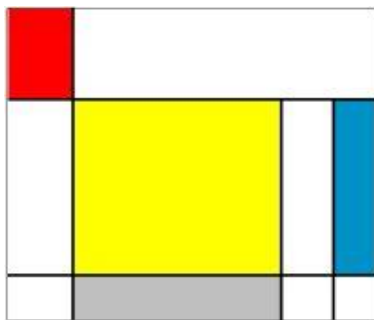
two



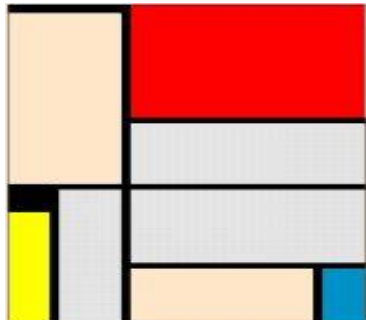
three



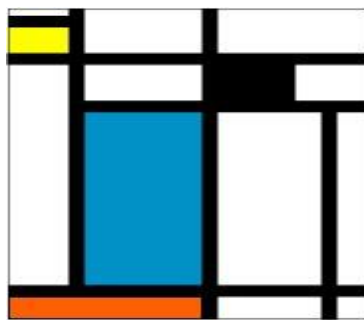
four



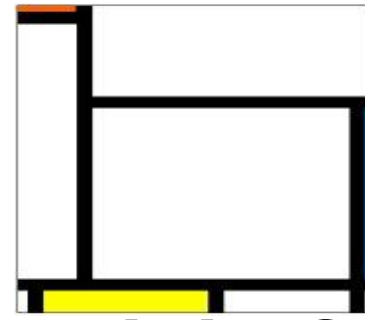
five



six



seven



eight ?

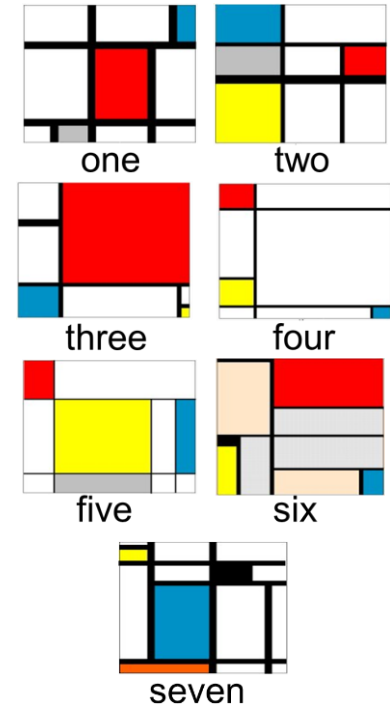


Pieter Cornelis  
Mondriaan  
(1872 – 1944)

# 데이터의 정의

- 아래와 같이 총 7개의 데이터가 존재한다고 가정하자.

| No. | Lines | Line types | Rectangles | Colors | Mondrian |
|-----|-------|------------|------------|--------|----------|
| #1  | 6     | 1          | 10         | 4      | No       |
| #2  | 4     | 2          | 8          | 5      | No       |
| #3  | 5     | 2          | 7          | 4      | Yes      |
| #4  | 5     | 1          | 8          | 4      | Yes      |
| #5  | 5     | 1          | 10         | 5      | No       |
| #6  | 6     | 1          | 8          | 6      | Yes      |
| #7  | 7     | 1          | 14         | 5      | No       |



- 이때, 새로운 인스턴스가 아래와 같이 주어졌을 때, 이는 어떻게 분류되는가?

| No. | Lines | Line types | Rectangles | Colors | Mondrian |
|-----|-------|------------|------------|--------|----------|
| NEW | 7     | 2          | 9          | 4      | ?        |



NEW ?

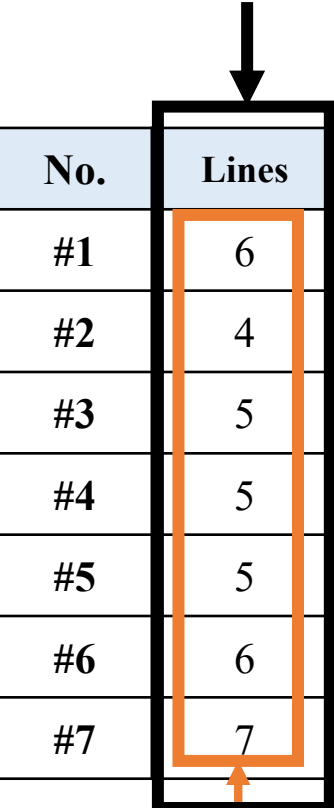
## 2. Data normalization

- 주어진 데이터셋에 대한 normalization을 수행한다.

$$x'_{i,t} = \frac{x_{i,t} - \bar{x}_t}{\sigma_t}$$

- $\bar{x}_t$ :  $t$ 번째 열(column)의 평균
  - $\sigma_t$ :  $t$ 번째 열(column)의 표준 편차
- 데이터셋을 왜 normalization 해야 하는가?
  - 데이터셋을 자세히 보면 각각의 feature (= column) 별로 값의 scale, 즉 분포가 확연하게 다른 것을 확인할 수 있다.
  - 따라서, 이를 평균을 0, 표준편차를 1로 조정하여 분포를 동일하게 맞추으로써 분석의 정확도를 높이기 위함이다.

$t^{\text{th}}$  attribute



| No. | Lines | Line types | Rectangles | Colors |
|-----|-------|------------|------------|--------|
| #1  | 6     | 1          | 10         | 4      |
| #2  | 4     | 2          | 8          | 5      |
| #3  | 5     | 2          | 7          | 4      |
| #4  | 5     | 1          | 8          | 4      |
| #5  | 5     | 1          | 10         | 5      |
| #6  | 6     | 1          | 8          | 6      |
| #7  | 7     | 1          | 14         | 5      |



$\bar{x}_t$



$\sigma_t$

## 2. Data normalization 수행

- Normalized dataset은 다음과 같다.

| No. | Lines  | Line types | Rectangles | Colors | Mondrian |
|-----|--------|------------|------------|--------|----------|
| #1  | 0.632  | -0.632     | 0.327      | -1.021 | No       |
| #2  | -1.581 | 1.581      | -0.588     | 0.408  | No       |
| #3  | -0.474 | 1.581      | -1.046     | -1.021 | Yes      |
| #4  | -0.474 | -0.632     | -0.588     | -1.021 | Yes      |
| #5  | -0.474 | -0.632     | 0.327      | 0.408  | No       |
| #6  | 0.632  | -0.632     | -0.588     | 1.837  | Yes      |
| #7  | 1.739  | -0.632     | 2.157      | 0.408  | No       |

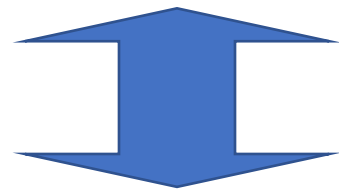
- 새로운 인스턴스에 대해서도 마찬가지로 normalization을 수행하면 다음과 같다.

| No. | Lines | Line types | Rectangles | Colors | Mondrian |
|-----|-------|------------|------------|--------|----------|
| NEW | 1.739 | 1.581      | -0.131     | -1.021 | ?        |

### 3. 인스턴스 간의 거리(유사도) 계산

- 이제, 새로운 인스턴스와 각각의 데이터 간의 거리를 계산한다.

| No. | Lines  | Line types | Rectangles | Colors | Mondrian |
|-----|--------|------------|------------|--------|----------|
| #1  | 0.632  | -0.632     | 0.327      | -1.021 | No       |
| #2  | -1.581 | 1.581      | -0.588     | 0.408  | No       |
| #3  | -0.474 | 1.581      | -1.046     | -1.021 | Yes      |
| #4  | -0.474 | -0.632     | -0.588     | -1.021 | Yes      |
| #5  | -0.474 | -0.632     | 0.327      | 0.408  | No       |
| #6  | 0.632  | -0.632     | -0.588     | 1.837  | Yes      |
| #7  | 1.739  | -0.632     | 2.157      | 0.408  | No       |



| No. | Lines | Line types | Rectangles | Colors | Mondrian |
|-----|-------|------------|------------|--------|----------|
| NEW | 1.739 | 1.581      | -0.131     | -1.021 | ?        |

$$D(x_q, x_1) = \sqrt{\sum_{t=1}^4 (x_{q,t} - x_{1,t})^2} = 2.517$$

$$D(x_q, x_2) = \sqrt{\sum_{t=1}^4 (x_{q,t} - x_{2,t})^2} = 3.644$$

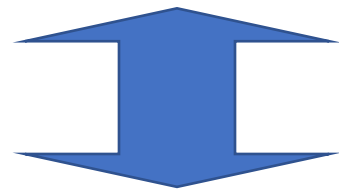
$$D(x_q, x_3) = \sqrt{\sum_{t=1}^4 (x_{q,t} - x_{3,t})^2} = 2.395$$

$$D(x_q, x_4) = \sqrt{\sum_{t=1}^4 (x_{q,t} - x_{4,t})^2} = 3.164$$

# 3. 인스턴스 간의 거리(유사도) 계산

- 이제, 새로운 인스턴스와 각각의 데이터 간의 거리를 계산한다.

| No. | Lines  | Line types | Rectangles | Colors | Mondrian |
|-----|--------|------------|------------|--------|----------|
| #1  | 0.632  | -0.632     | 0.327      | -1.021 | No       |
| #2  | -1.581 | 1.581      | -0.588     | 0.408  | No       |
| #3  | -0.474 | 1.581      | -1.046     | -1.021 | Yes      |
| #4  | -0.474 | -0.632     | -0.588     | -1.021 | Yes      |
| #5  | -0.474 | -0.632     | 0.327      | 0.408  | No       |
| #6  | 0.632  | -0.632     | -0.588     | 1.837  | Yes      |
| #7  | 1.739  | -0.632     | 2.157      | 0.408  | No       |



| No. | Lines | Line types | Rectangles | Colors | Mondrian |
|-----|-------|------------|------------|--------|----------|
| NEW | 1.739 | 1.581      | -0.131     | -1.021 | ?        |

$$D(x_q, x_5) = \sqrt{\sum_{t=1}^4 (x_{q,t} - x_{5,t})^2} = 3.472$$

$$D(x_q, x_6) = \sqrt{\sum_{t=1}^4 (x_{q,t} - x_{6,t})^2} = 3.808$$

$$D(x_q, x_7) = \sqrt{\sum_{t=1}^4 (x_{q,t} - x_{7,t})^2} = 3.490$$

## 4. 입력 인스턴스와 가장 가까운 K개의 인스턴스 선택

- 앞서 거리를 계산한 결과를 토대로 입력 인스턴스와 거리가 가장 가까운(또는 가장 유사도가 높은) 순서대로 데이터를 정렬한다.

| No. | Distance | Mondrian |
|-----|----------|----------|
| #1  | 2.517    | No       |
| #2  | 3.644    | No       |
| #3  | 2.395    | Yes      |
| #4  | 3.164    | Yes      |
| #5  | 3.472    | No       |
| #6  | 3.808    | Yes      |
| #7  | 3.490    | No       |

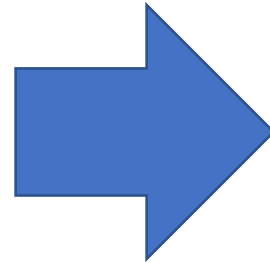


| No. | Distance | Mondrian |
|-----|----------|----------|
| #3  | 2.395    | Yes      |
| #1  | 2.517    | No       |
| #4  | 3.164    | Yes      |
| #5  | 3.472    | No       |
| #7  | 3.490    | No       |
| #2  | 3.644    | No       |
| #6  | 3.808    | Yes      |

# 5. K개의 인스턴스로부터 입력 인스턴스의 클래스 분류

- K개의 인스턴스를 가장 많이 포함하는 클래스가 무엇인지 파악한다.
- 이를 입력 인스턴스의 최종 클래스로 분류한다.

| No. | Distance | Mondrian |
|-----|----------|----------|
| #3  | 2.395    | Yes      |
| #1  | 2.517    | No       |
| #4  | 3.164    | Yes      |
| #5  | 3.472    | No       |
| #7  | 3.490    | No       |
| #2  | 3.644    | No       |
| #6  | 3.808    | Yes      |



## 분류 결과

- 1-NN → Yes
- 3-NN → Yes
- 5-NN → No

| No. | Lines | Line types | Rectangles | Colors | Mondrian |
|-----|-------|------------|------------|--------|----------|
| NEW | 1.739 | 1.581      | -0.131     | -1.021 | ?        |

# K-NN algorithm의 주요 이슈

# K-NN algorithm의 주요 이슈

- 인스턴스 간의 거리를 계산하는 방법
  - 유클리드 거리 (Euclidean distance)
  - 맨하탄 거리 (Manhattan distance)
  - 코사인 유사도 (Cosine similarity)
  - 피어슨 상관 계수 (Pearson correlation coefficient)
- $k$ 개의 근접 인스턴스로부터 입력 인스턴스  $\vec{x}^{(q)}$ 의 클래스를 결정하는 방법
  - Discrete case인 경우(classification): **Voting**
  - Continuous case인 경우(regression): **Average**

# 인스턴스 간의 거리(유사도)를 계산하는 방법

| 유사도 계산 방법 | 입력 인스턴스 $\vec{x}^{(q)}$ 와 $i$ 번째 인스턴스 $\vec{x}^{(i)}$ 간의 거리(유사도) 계산 공식                                                                                                                                                                     |
|-----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 유클리드 거리   | $D_E(\vec{x}^{(q)}, \vec{x}^{(i)}) = \sqrt{\sum_{j=1}^n (x_j^{(q)} - x_j^{(i)})^2}$                                                                                                                                                        |
| 맨하탄 거리    | $D_M(\vec{x}^{(q)}, \vec{x}^{(i)}) = \sum_{j=1}^n  x_j^{(q)} - x_j^{(i)} $                                                                                                                                                                 |
| 코사인 유사도   | $SIM_C(\vec{x}^{(q)}, \vec{x}^{(i)}) = \frac{\vec{x}^{(q)} \cdot \vec{x}^{(i)}}{\ \vec{x}^{(q)}\  \ \vec{x}^{(i)}\ } = \frac{\sum_{j=1}^n x_j^{(q)} x_j^{(i)}}{\sqrt{\sum_{j=1}^n x_j^{(q)2}} \sqrt{\sum_{j=1}^n x_j^{(i)2}}}$             |
| 피어슨 상관 계수 | $PCC(\vec{x}^{(q)}, \vec{x}^{(i)}) = \frac{\sum_{j=1}^n (x_j^{(q)} - \mu(\vec{x}^{(q)})) (x_j^{(i)} - \mu(\vec{x}^{(i)}))}{\sqrt{\sum_{j=1}^n (x_j^{(q)} - \mu(\vec{x}^{(q)}))^2} \sqrt{\sum_{j=1}^n (x_j^{(i)} - \mu(\vec{x}^{(i)}))^2}}$ |

# 인스턴스의 클래스를 결정하는 방법

- 인스턴스의 label이 **discrete value**인 경우 (▶ 앞서 설명한 내용)
  - 각각의 인스턴스의 label이 discrete value → 클래스 번호(class index)
  - 즉, 각각의 인스턴스는 총  $T$ 개의 클래스 중 하나에 포함된 경우이다.
  - 이때,  $f : \mathbb{R}^n \rightarrow V$  where  $V \in \{1, 2, \dots, T\}$ .
- 그렇다면, 새로운 인스턴스의 클래스는 어떻게 할당하는가?
  - $k$ 개의 인스턴스 각각이 어떤 클래스에 속하는지 파악한다.
  - 이때,  $k$ 개의 인스턴스를 가장 많이 포함하는 클래스를 새로운 인스턴스의 클래스로 지정한다.

# 인스턴스의 클래스를 결정하는 방법

- 인스턴스의 label이 continuous value인 경우
  - 즉, 각각의 인스턴스의 label이 실수인 경우로, 이때  $f : \mathbb{R}^n \rightarrow \mathbb{R}$  이다.
- 이 경우, 새로운 인스턴스의 label은 어떻게 계산되는가?
  - $k$ 개의 인스턴스 각각의 label 값들의 평균을 계산하고, 이를 새로운 인스턴스의 label로 지정한다.

$$f(\vec{x}^{(q)}) = \frac{1}{k} \sum_{j=1}^k f(\vec{x}^{(j)})$$



거리(유사도)에 따라 다른 가중치를 부여하여 계산하는 방법도 널리 활용됨

- 따라서, 이 경우는 분류(classification) 문제가 아닌 **회귀(regression) 문제**의 범주에 포함된다.

# 학습의 관점에서 K-NN algorithm의 의미는?

- K-NN algorithm이 학습하는 것은 target function  $f$ 의 근사 함수이다.
- 주어진 데이터셋의 label이 discrete value인 경우, target function  $f$ 는 다음과 같이  $\hat{f} : \mathbb{R}^m \rightarrow V$ 의 근사 함수로 근사하여 표현할 수 있다.

$$\hat{f}(\vec{x}^{(q)}) \leftarrow \arg \max_v \sum_{i=1}^k \delta(v, f(\vec{x}^{(i)}))$$

- 한편, 데이터셋의 label이 continuous value인 경우, target function  $f$ 는 다음과 같이 근사 함수  $\hat{f} : \mathbb{R}^n \rightarrow \mathbb{R}$ 로 표현할 수 있다.

$$\hat{f}(\vec{x}^{(q)}) \leftarrow \frac{1}{k} \sum_{i=1}^k f(\vec{x}^{(i)})$$

# Q & A